



UNIVERSIDAD PONTIFICIA DE COMILLAS (ICAI)

DEGREE IN TELECOMMUNICATIONS ENGINEERING

MASTERS THESIS (APPENDIX B)

Line-Rate DDoS Detection System Using DPDK and OctoSketch

Author:
Claudio Esteban Quesada

Supervisor:
Zaoxing Liu

Author's signature:

Supervisor's signature:

29-01-2026

Índice

- 1 Introduction 2
- 2 State of the Art 2
 - 2.1 Traditional DDoS Detection Approaches 2
 - 2.2 Kernel Bypass and High-Speed Packet Processing 3
 - 2.3 Probabilistic Data Structures for Traffic Monitoring 3
 - 2.4 Machine Learning for Embedded DDoS Detection 3
- 3 Motivation 3
- 4 Objectives 4
- 5 Methodology 5
 - 5.1 Experimental Setup 5
 - 5.2 Feature Extraction Strategies 5
 - 5.3 Dataset Generation and Model Training 6
 - 5.4 Embedded Inference and Detection Logic 6
- 6 Planning and Next Steps 7
- 7 Resources 8
 - 7.1 Hardware Infrastructure 8
 - 7.2 Software Components 8
 - 7.3 Datasets 9
 - 7.4 Development Tools 9

1 Introduction

Distributed Denial of Service (DDoS) attacks remain one of the most disruptive threats to modern network infrastructures. The continuous increase in bandwidth availability, the proliferation of botnets, and the commoditization of attack services have led to a steady rise in volumetric and amplification-based attacks. Contemporary attacks frequently reach tens of gigabits per second and may overwhelm network services within milliseconds.

Traditional detection systems deployed in general-purpose operating systems face fundamental performance limitations. Packet processing inside the Linux kernel typically relies on interrupt-driven mechanisms and per-packet processing paths that limit throughput to only a few million packets per second. This becomes insufficient in high-speed environments (25 Gbps and above), where tens of millions of packets per second must be processed continuously and deterministically.

Most modern machine learning-based DDoS detection systems rely on features derived from Deep Packet Inspection (DPI). However, the widespread adoption of encrypted protocols such as TLS 1.3, QUIC, and encrypted DNS significantly reduces payload visibility, limiting the effectiveness of DPI-dependent approaches.

This project develops a line-rate DDoS detection system based on ML-based classification, integrated directly into a high-performance packet processing architecture. The system supports different feature extraction strategies, including approaches that rely on deep packet inspection as well as approaches based exclusively on packet headers and probabilistic sketches without inspecting payload contents.

The objective is to evaluate whether header-based probabilistic features can provide classification accuracy comparable to payload-dependent approaches while offering superior scalability and compatibility with encrypted traffic environments.

The final system performs classification exclusively through trained machine learning models. Statistical counters are maintained for monitoring purposes but do not influence detection decisions.

2 State of the Art

2.1 Traditional DDoS Detection Approaches

Conventional DDoS detection mechanisms can be categorized into threshold-based anomaly detection, signature-based systems, and machine learning-based classification.

Threshold-based systems compare traffic metrics against static values, while signature-based systems rely on known attack patterns. Machine learning approaches use traffic-derived features to classify benign and malicious behavior.

While ML-based systems provide strong classification accuracy, many rely heavily on features derived from deep packet inspection. As encrypted traffic becomes dominant, payload-dependent features lose reliability, creating a structural limitation for DPI-dependent machine learning models.

2.2 Kernel Bypass and High-Speed Packet Processing

High-speed packet processing frameworks such as DPDK enable direct access to NICs from user space using poll-mode drivers, hugepages, and zero-copy buffers. This architecture eliminates kernel overhead and enables deterministic packet processing at line rate, making it suitable for real-time detection in 25 Gbps environments.

2.3 Probabilistic Data Structures for Traffic Monitoring

Tracking per-IP statistics at high speed introduces memory scalability challenges. The Count-Min Sketch (CMS) provides a probabilistic alternative with constant memory usage and $O(1)$ updates.

In addition to global traffic sketches, probabilistic structures can be instantiated per protocol to capture protocol-specific traffic distributions. This enables discrimination between attack vectors using header-level information without requiring access to payload contents.

2.4 Machine Learning for Embedded DDoS Detection

Recent works demonstrate that supervised classifiers can achieve strong performance in DDoS classification tasks when provided with well-engineered traffic features.

However, many ML-based approaches assume the availability of DPI-derived features. With the growth of encrypted protocols, payload visibility is reduced, limiting the applicability of such methods. This motivates the exploration of header-only feature extraction strategies compatible with encrypted traffic environments.

3 Motivation

The motivation of this project arises from structural limitations observed in current high-speed DDoS detection systems, particularly when deployed in modern network environments characterized by high throughput and increasing traffic encryption.

First, **scalability** remains a fundamental constraint. Detection systems operating in 25 Gbps environments must process tens of millions of packets per second while maintaining deterministic latency and avoiding contention between processing threads. Kernel-based packet processing architectures introduce interrupt overhead and context switching costs that prevent sustainable line-rate monitoring. A viable detection system must therefore integrate directly into a high-performance data plane architecture capable of sustained throughput under heavy load.

Second, **payload visibility is progressively decreasing**. Many state-of-the-art machine learning approaches depend on features extracted via Deep Packet Inspection (DPI), including protocol-specific fields, application-layer indicators, and content-derived statistics. However, the rapid adoption of encrypted protocols such as TLS 1.3, QUIC, and encrypted DNS significantly reduces the availability of payload information. In encrypted environments, DPI-derived features may become unavailable or unreliable, limiting the practical applicability of payload-dependent detection models.

This limitation motivates a critical question: whether features extracted exclusively from packet headers and probabilistic traffic summaries can provide sufficient discriminative power for accurate multi-class DDoS classification. If header-only features can achieve comparable accuracy to DPI-based feature sets, detection systems could remain effective even in fully encrypted network environments.

Third, **deployment constraints** require that classification logic be embedded directly within the packet processing pipeline. External inference services introduce latency and architectural complexity that are incompatible with real-time line-rate detection. A practical solution must therefore rely on ML-based classification integrated natively within the data plane.

The objective of this project is to design an ML-based detection architecture capable of operating with both payload-dependent and header-only feature extraction strategies, and to evaluate the viability of header-only probabilistic representations in encrypted traffic scenarios.

The final system performs classification exclusively through trained machine learning models. Statistical counters provide complementary monitoring and diagnostic information but do not influence alert decisions.

4 Objectives

The primary objective of this project is to design and implement a line-rate ML-based DDoS detection system capable of operating in high-speed and partially encrypted network environments while maintaining competitive classification performance.

The specific objectives of the project are:

- Design a scalable multi-core DPDK-based packet processing architecture capable of sustained line-rate operation in 25 Gbps environments.
- Implement probabilistic traffic monitoring using sketch-based data structures to efficiently summarize high-cardinality traffic patterns with bounded memory usage.
- Engineer structured traffic features derived from both payload-aware extraction strategies and header-only probabilistic representations.

- Train supervised machine learning models for multi-class DDoS attack classification using structured detection-window datasets.
- Evaluate and compare feature extraction strategies with and without payload inspection to determine whether header-only features can achieve classification performance comparable to DPI-dependent approaches.
- Integrate model inference directly into the packet processing pipeline, ensuring real-time decision-making without external processing services.
- Validate the system in a controlled multi-node CloudLab environment under reproducible traffic generation scenarios.

These objectives combine architectural scalability, encrypted-traffic compatibility, and embedded ML deployment into a unified experimental framework.

5 Methodology

This section describes the methodological framework used to design, implement, and evaluate the proposed DDoS detection system. The methodology integrates high-performance packet processing, probabilistic monitoring, structured feature engineering, and embedded machine learning inference.

5.1 Experimental Setup

Experiments are conducted in a controlled multi-node CloudLab deployment designed to reproduce realistic high-speed network conditions. The testbed includes dedicated traffic generation nodes, a monitoring node running the DPDK-based detector, and a victim server interconnected through high-speed links.

Traffic is generated using pre-built PCAP datasets representing benign background traffic and multiple DDoS attack types. Each experimental run follows a structured timeline including baseline benign traffic, attack injection, and recovery phases. This design enables systematic dataset generation and reproducible evaluation of classification performance.

5.2 Feature Extraction Strategies

The system supports multiple feature extraction configurations.

One configuration incorporates deep packet inspection combined with probabilistic sketch-based traffic summaries. Another configuration relies exclusively on packet header information and per-protocol sketch instances, without accessing packet payload contents.

This methodological design enables direct comparison between payload-dependent and header-

only approaches, allowing evaluation of their relative classification performance and operational scalability.

5.3 Dataset Generation and Model Training

Detection operates over fixed time windows. For each window, cumulative traffic statistics are aggregated and transformed into structured feature vectors capturing volumetric, protocol-related, temporal, and distribution-based characteristics.

Datasets are constructed by aggregating multiple experimental runs under different traffic conditions. Feature normalization is performed using standard scaling techniques to ensure consistent model training.

Supervised machine learning models are trained for multi-class classification, distinguishing between benign traffic and multiple attack categories.

5.4 Embedded Inference and Detection Logic

The trained model is exported and loaded directly into the DPDK-based detector through a native C-based inference interface. Inference is executed once per detection window on a coordination core, ensuring real-time classification without external services.

Classification decisions are determined exclusively by the trained machine learning model. Statistical counters remain available for monitoring and diagnostic purposes but do not influence alert decisions.

This architecture ensures that detection remains fully embedded within the data plane while supporting both payload-aware and header-only feature extraction strategies.

6 Planning and Next Steps

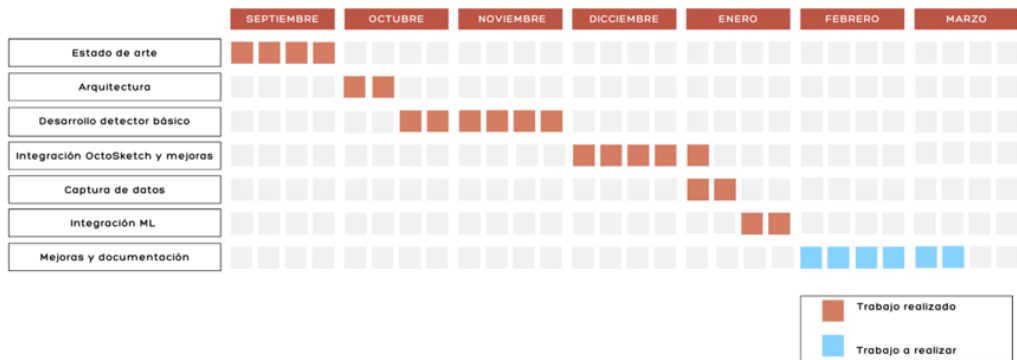


Figure 6.1: Project planning timeline from September to March.

Figure 6.1 summarizes the progression of the project, structured into seven work blocks developed incrementally from September through March. The planning reflects a gradual transition from conceptual foundations to system implementation, experimental validation, and final consolidation.

Block A – State of the Art (September). This initial phase focused on a comprehensive literature review covering DDoS detection approaches, kernel bypass frameworks, probabilistic data structures, and machine learning-based classification in high-speed network environments. This stage established the theoretical and methodological foundation of the project.

Block B – Architecture Design (October). During this phase, the multi-core DPDK-based packet processing architecture was designed. This included the definition of RSS-based traffic distribution, per-worker data structure organization, and the coordination logic required for feature aggregation and classification.

Block C – Basic Detector Development (October–November). The core detection pipeline was implemented, including packet parsing, protocol-level counters, and the initial monitoring framework. This phase established the functional baseline of the system before the integration of probabilistic structures.

Block D – OctoSketch Integration and Enhancements (November–December). Probabilistic sketch-based data structures were integrated to enable scalable per-IP traffic monitoring. The architecture was extended to support per-protocol sketch instances, allowing feature extraction based solely on packet headers without payload inspection.

Block E – Data Capture (December). Systematic data collection was performed using struc-

tured experimental runs in CloudLab. Multiple attack types were replayed under controlled conditions following a baseline–attack–recovery protocol, enabling the generation of labeled multi-class datasets for supervised learning.

Block F – ML Integration (January). Supervised machine learning models were trained using the collected datasets. The trained model was exported and embedded directly into the DPDK-based detector through a native C inference interface, enabling real-time classification at the level of detection windows.

Block G – Improvements and Documentation (January–March). The final phase includes refinement of feature extraction strategies, comparative evaluation across payload-dependent and header-only approaches, performance analysis, and preparation of the final thesis documentation.

Together, these blocks illustrate a structured progression from theoretical grounding to architectural design, implementation, experimental validation, and final consolidation.

7 Resources

7.1 Hardware Infrastructure

Experimental validation is conducted on CloudLab high-performance nodes equipped with multi-core CPUs and 25 Gbps network interface cards. Hugepage-enabled memory allocation is used to support DPDK-based packet processing.

This infrastructure enables sustained high-throughput experimentation under controlled and reproducible conditions.

7.2 Software Components

The system integrates multiple software components across different architectural layers:

- DPDK framework for high-speed user-space packet processing.
- C language for implementation of the detector and monitoring pipeline.
- Python for dataset preparation, feature engineering, and model training.
- Machine learning libraries for supervised multi-class classification.
- Scikit-learn utilities for preprocessing and evaluation.

7.3 Datasets

Attack scenarios are designed following the methodological principles of the CIC-DDoS-2019 dataset, covering volumetric and amplification-based attacks across multiple protocols. Custom PCAP traces are generated and replayed under controlled conditions to construct structured multi-class datasets suitable for supervised learning.

7.4 Development Tools

Development relies on standard software engineering tools, including:

- GCC compiler toolchain
- CMake build system
- Git version control
- Structured logging and feature extraction utilities